Mini Project

Department Budget Tracker

You manage budget data for 4 departments in a spreadsheet. Every quarter you manually calculate totals, flag over-budget departments, and find the biggest spender. In Excel, you use manual steps like SUMIF, MAX, MATCH, conditional formatting etc.

Build a Python script that analyses department expense data using different functions. Each function replaces a task you currently perform manually in Excel.

Step_1: Copy the data below exactly as given below.

```
expenses = [
    {"dept": "Marketing",   "category": "Ads",      "amount": 52000, "month": "Jan"},
    {"dept": "Marketing",   "category": "Events",   "amount": 38000, "month": "Feb"},
    {"dept": "Engineering", "category": "Software", "amount": 91000, "month": "Jan"},
    {"dept": "Engineering", "category": "Hardware", "amount": 47000, "month": "Feb"},
    {"dept": "HR",          "category": "Training", "amount": 23000, "month": "Jan"},
    {"dept": "HR",          "category": "Hiring",   "amount": 61000, "month": "Feb"},
    {"dept": "Sales",       "category": "Travel",   "amount": 34000, "month": "Jan"},
    {"dept": "Sales",       "category": "Gifts",    "amount": 18000, "month": "Feb"},
]

budgets = {
    "Marketing":   80000,
    "Engineering": 120000,
    "HR":          70000,
    "Sales":       60000,
}
```

Step_2: Create a function called **`total_by_dept(expenses)`**, which takes in the expenses list and returns a dictionary called **`totals`**.

Your output should look like:

`{'Marketing': 90000, 'Engineering': 138000, 'HR': 84000, 'Sales': 52000}`

Step_3: Create a function called **`budget_status(totals, budgets)`**, which takes two parameters: **`totals`** (result of Function 1) and budgets (the dictionary above). This function should compute the gap between amount spent and budget limit.

If gap is greater than 0, print a warning line that includes OVER BUDGET and the gap amount.

Otherwise, print a confirmation line showing the remaining amount.

This function does not return anything, it only prints.

Step_4: Create another function called **`highest_spender(totals)`**, which takes in **`totals`** (result of Function 1) and returns both the department name and its total spend for the department which spent the most.

[Hint: Use Python's built-in **`max()`** function with a lambda as the key argument. The lambda must tell **`max()`** to compare departments by their spend value, not by name alphabetically.]

Step_5 (Connect everything in **`main()`** function):

Write a function called **`main()`** that calls each of the four functions and prints the full report. Call **`main()`** at the bottom of your file to run it.

Note:

Instead of calling **`main()`** *directly, Python developers often write*

```
if __name__ == "__main__":
    main()
```

at the bottom. This is simply a safer way to run your script. This ensures your script runs only when you execute it directly, not when another Python file imports it.

Expected Output

When you run your script, the output must match the following exactly.

```
--- Budget Status ---
Marketing   : ₹90,000 spent of ₹80,000  ⚠️ OVER BUDGET by ₹10,000
Engineering : ₹138,000 spent of ₹120,000  ⚠️ OVER BUDGET by ₹18,000
HR          : ₹84,000 spent of ₹70,000  ⚠️ OVER BUDGET by ₹14,000
Sales       : ₹52,000 spent of ₹60,000  ✓ Under budget  (₹8,000 remaining)

===== DEPARTMENT BUDGET REPORT =====

--- Total Spend by Department ---
Marketing   :  ₹90,000
Engineering :  ₹138,000
HR          :  ₹84,000
Sales       :  ₹52,000

--- Budget Status ---
Marketing   : ₹90,000 spent of ₹80,000  ⚠️ OVER BUDGET by ₹10,000
Engineering : ₹138,000 spent of ₹120,000  ⚠️ OVER BUDGET by ₹18,000
HR          : ₹84,000 spent of ₹70,000  ⚠️ OVER BUDGET by ₹14,000
Sales       : ₹52,000 spent of ₹60,000  ✓ Under budget  (₹8,000 remaining)

--- Highest Spender ---
Engineering spent the most: ₹138,000
```

If you have completed this exercise successfully, you have taken a significant step, your functions are modular, reusable, and can be applied to any similar dataset simply by changing the data at the top.

In the upcoming chapters, you will learn to read this data directly from CSV and Excel files, eliminating the need to hardcode it, and later use the ***pandas*** *library to perform the same analysis in a fraction of the time.*

The goal of this exercise is to test your understanding. You have now seen firsthand how Python can replace repetitive manual work in Excel with logic you write once and reuse always.